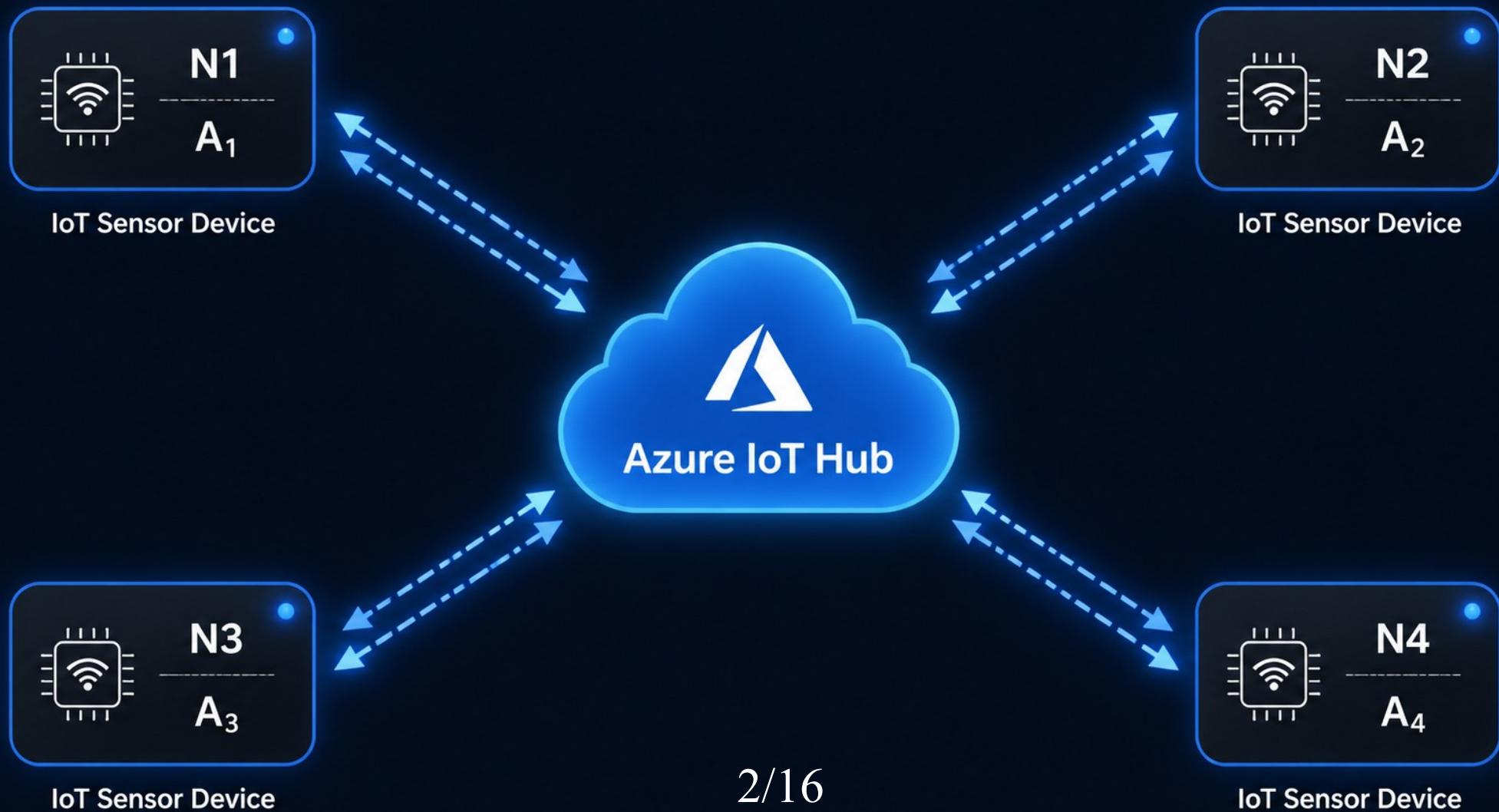


Множење матрица – Парадигма дистрибуираног рачунања, технологија сензорских мрежа



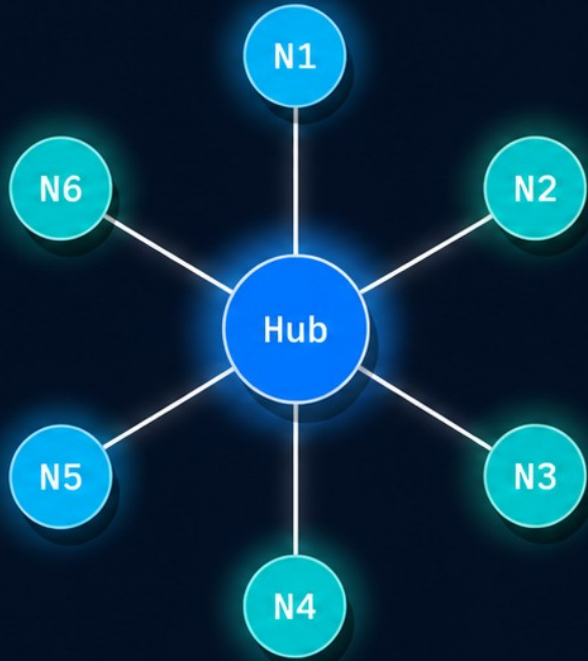
Електротехнички факултет,
Универзитет у Београду



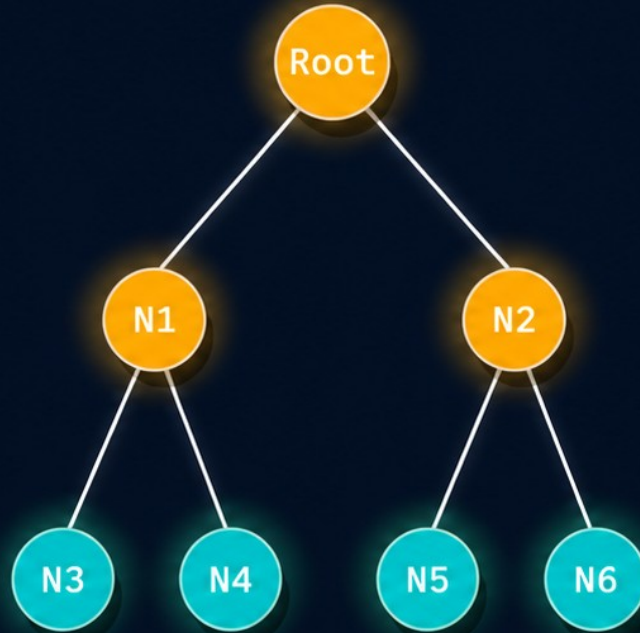
Шта је WSN?

- WSN – скуп географски распоређених сензорских чворова:
 - Чворови комуницирају бежично
 - Сваки чвор има:
 - MCU (ARM Cortex-M/ESP32), Радио модул, Батерију
 - Протоколи:
 - MQTT – најчешћи, Zigbee, LoRa, WiFi
 - Ограничења:
 - CPU ~ 16-256 MHz
 - RAM 2-512 KB
 - ~10 mW активан

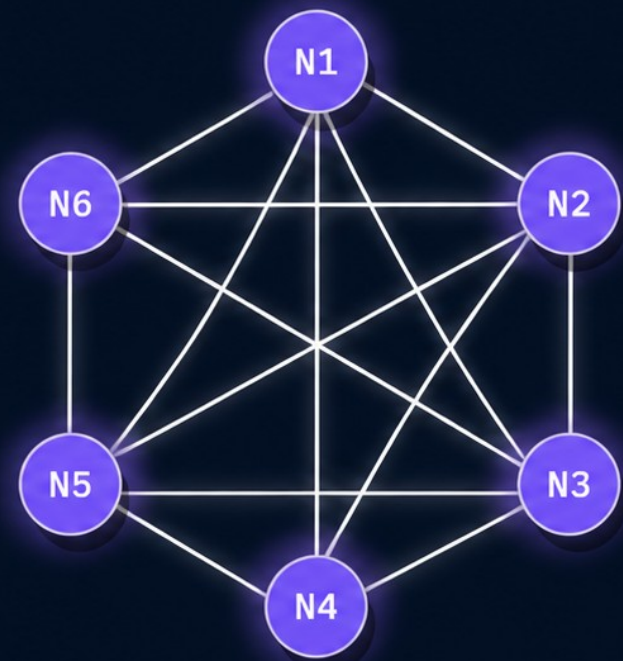
Star Topology



Tree Topology



Mesh Topology



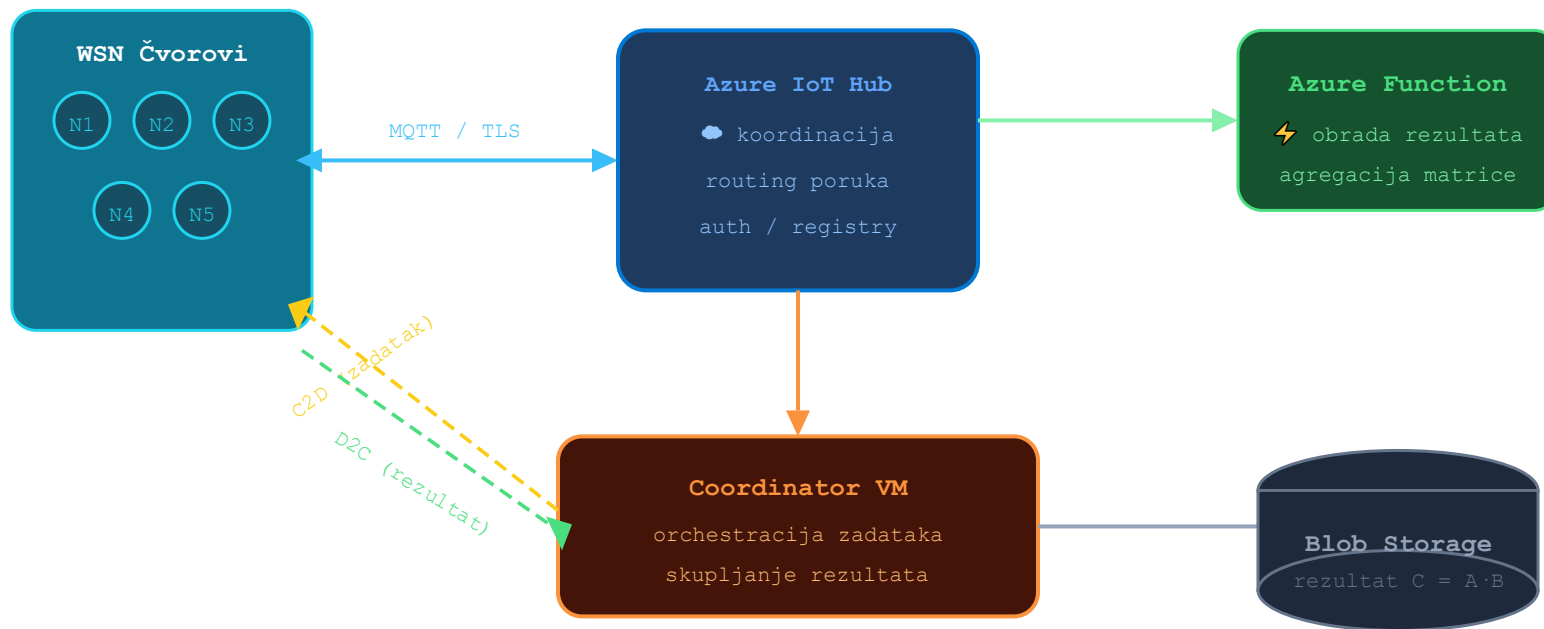
Проблем и мотивација

- $A(m \times k) \times B(k \times n) \rightarrow O(m \cdot k \cdot n)$ операција – скупо за веће димензије
- Један чвор не може учитати матрицу $> 100 \times 100$ у RAM
- Решење: подели A по редовима на N чворова
- Чвор i рачуна: $C_i = A_i \times B$ локално
- Master скупља све $C_i \rightarrow$ реконструира C

WSN vs IoT

	WSN	IoT
Фокус	Чворови међусобно комуницирају и заједно рачунају	Уређаји шаљу податке у cloud који обрађује
Комуникација	Node-to-node, multi-hop, mesh routing	Device-to-Cloud једносмерно/бидирекционо
Cloud	Опционалан или нема	Централан, обавезан
Протокол	Zigbee, LEACH routing, gossip	MQTT/HTTP ка серверу
Улога чвора	Активни учесник у рачунању	Пасивни скупљач података
Топологија	Peer-to-peer, нема нужно централног чвора	Hub-and-spoke, увек постоји сервер

Azure arhitektura sistema



Azure архитектура система

- Azure IoT Hub – Централни хаб, регистар уређаја, C2D/D2C routing
- Direct Methods – синхрони RPC позиви Cloud → Чвор (timeout 60s)
- Azure VM/Container – покреће coordinator логику
- Azure Blob Storage – чување улазних/излазних матрица

Ток података

- Координатор приима улазне матрице A и B
- Дели A на N партиција: $A_0, A_1, \dots, A_{N-1}$
- Паралелно шаље Direct Method 'multiply' payload: $\{A_i, B, \text{row_start}, \text{row_end}\}$
- Чвор прима, множи $C_i = A_i \times B$ локално
- Чвор враћа `MethodResponse` са C_i и метаподацима
- Координатор чека све одговоре (`asyncio.gather`)
- Реконструише: $C[\text{row_start}:\text{row_end}] = C_i$
- Враћа финалну матрицу C

Координатор – код,
кључни делови

```
def partition_matrix(A, n_workers):
    chunk = A.shape[0] // n_workers
    return [(i*chunk, (i+1)*chunk, A[i*chunk:(i+1)*chunk].tolist())
            for i in range(n_workers)]

def send_task(registry_manager, device_id, task):
    method = CloudToDeviceMethod(method_name="multiply",
                                   payload=task,
                                   response_timeout_in_seconds=60)
    return registry_manager.invoke_device_method(device_id,
method).payload

async def distributed_matmul(A, B):
    parts = partition_matrix(A, len(WORKER_DEVICE_IDS))
    futures = [loop.run_in_executor(None, send_task, reg, dev, payload)
               for dev, payload in zip(WORKER_DEVICE_IDS, parts)]
    responses = await asyncio.gather(*futures) # paralelno ceka sve
    C = np.zeros((A.shape[0], B.shape[1]))
    for resp in responses:
        C[resp["row_start"]:resp["row_end"]] = np.array(resp["C_chunk"])
    return C
```

```

def local_matmul(A_chunk, B):
    A = np.array(A_chunk, dtype=np.float32)
    return (A @ np.array(B, dtype=np.float32)).tolist() # numpy BLAS

def multiply_handler(method_request):
    p = method_request.payload
    t0 = time.time()
    C_chunk = local_matmul(p["A_chunk"], p["B"])
    resp = {"row_start": p["row_start"], "row_end": p["row_end"],
           "C_chunk": C_chunk, "compute_time": time.time() - t0}
    return MethodResponse.create_from_method_request(method_request, 200,
resp)

# Setup
client = IoTHubDeviceClient.create_from_connection_string(CONN_STR)
client.connect()
client.on_method_request_received = multiply_handler # registruj handler

```

Радник - код

Benchmark

димензије	секвенцијално(ms)	дистрибуирано(ms)	убрзање
50x50 x 50x50	0.69	16.85	0.04x
100x100 x 100x100	0.03	19.53	0.00x
200x200 x 200x200	3.88	19.8	0.2x
50x50 x 50x50	2.19	20.02	0.11x

- Overhead ~15ms/чвор доминира за мале матрице

Azure IoT Hub Setup

- Azure портал → Create IoT Hub (Free Tier)
- IoT devices → Add device → Device ID: wsn-node-01
- Копирај Primary Connection String за сваки чвор
- На чвору: `pip install azure-iot-device`, унеси connection string
- Coordinator: `pip install azure-iot-hub`

```
reg = IoTHubRegistryManager.from_connection_string(CONN_STR)
```

```
reg.create_device_with_sas(device_id="wsn-node-01", primary_key=None, status="enabled")
```

Изазови и ограничења

Изазов	Проблем	Решење
Латенција мреже	~50-200ms MQTT round-trip	Batch задаци, async gather
Меморија чвора	Max ~512KB RAM	Timeout + coded computation
Поузданост	Губитак пакета у WSN	MQTT QoS 1, retry
Overhead порука	JSON серијализација	MessagePack / бинарни формат

Закључак

- WSN + Azure IoT Hub = функционалан дистрибуирани matmul на edge чворовима
- Убрзање је реално тек за велике матрице
- Кључни overhead: JSON серијализација + MQTT латенција
- Даљи правци: Coded Distributed Computing, MPI-over-MQTT

Хвала на пажњи!

Припремили:

Павле Шаренац 2025/3068

sp253068m@student.etf.bg.ac.rs

Никола Николић 2025/3109

nn253109m@student.etf.bg.ac.rs